Пособие по разработке настольных приложений на базе WPF

Шпалитов А.Э.

Содержание

ВВЕДЕНИЕ 5

ВЕРСТКА XML 6

База 6
Интерактивные и не очень элементы 8
TextBlock 8
TextBox 8
Button 9
ComboBox 9
DatePicker 11
Контейнеры 12
Grid 12
StackPanel 17
DockPanel 18
WrapPanel 20
Списки 21
DataGrid 21
ListView (требует доработки) 24

НАВИГАЦИЯ СТРАНИЦАМИ 27

Навигация внутри страницы 28
Обратная навигация 28

ПОДКЛЮЧЕНИЕ БД 29

Создание модели 29
Создание единой переменной для связи с БД 29
Обновление модели 30

ДОБАВЛЕНИЕ ДАННЫХ В БД 31

РЕАЛИЗАЦИЯ ФОРМЫ ЧЕРЕЗ BINDING 33

ВАЛИДАЦИЯ СВОЙСТВ ОБЪЕКТА 38

Реализация валидации 38
Проверки на валидность 39
Проверка на пустоту строкового свойства 39

Проверка на длину строкового свойства 39
Проверка числового свойства 40
Проверка на свойства связанных таблиц 40
Проверка даты 40
Проверка через регулярные выражения (скоро) 41

LINQ И ЛЯМБДА ВЫРАЖЕНИЯ (СКОРО) 42

LINQ 42
Лямбда-выражения 44
Основные методы LINQ 45
FirstOrDefault 46
LastOrDefault 46
Where 47
OrderBy/OrderByDescending/ThenBy/ThenByDescending 47
Sum 48
Average 48
Count 49
Select 49
All 49
Any 49
GroupBy 49
ToDictionary 49

СТИЛИЗАЦИЯ ИНТЕРФЕЙСА ПРИЛОЖЕНИЯ (НЕСКОРО) 50

РАЗРАБОТКА БАЗЫ ДАННЫХ В MS SQL 51

Создание диаграммы 51
Добавление таблицы на диаграмму 53
Создание связей между таблицами 55
Создание связи “Много ко Многим” 58

GIT 61

Установка Git 61
Работа с Git Bash 61
Основные команды 61
Клонирование репозитория 61
Инициализация репозитория 62
Отправка изменений 62
Личные рекомендации 62

РАБОТА С GIT ЧЕРЕЗ VISUAL STUDIO **63**
РАБОТА С GIT ЧЕРЕЗ GITHUB DESKTOP (НЕСКОРО) **63**

ЧАСТО ВСТРЕЧАЕМЫЕ ВОПРОСЫ И ПРОБЛЕМЫ **64**

Введение

В мире разработки программного обеспечения существует множество технологий и инструментов, позволяющих создавать настольные приложения. Одной из таких технологий является Windows Presentation Foundation (WPF) — мощная платформа, предоставляющая разработчикам широкие возможности для создания современных и удобных в использовании приложений под операционную систему Windows.

WPF выделяется среди других технологий благодаря своим мощным средствам для работы с графикой, анимацией и пользовательским интерфейсом. Она позволяет создавать интерфейсы, которые не только удобны в использовании, но и визуально привлекательны, обеспечивая высокий уровень интерактивности и гибкости.

Цель данного пособия — познакомить с основами и продвинутыми возможностями разработки приложений на базе WPF. Мы рассмотрим базовые концепции, такие как работа с XAML (eXtensible Application Markup Language), создание и управление окнами, элементы управления и их свойства. Также мы обратим внимание на архитектурные подходы, такие как Model-View-ViewModel (MVVM), и методы разработки гибких и масштабируемых приложений.

В процессе изучения вы научитесь создавать и настраивать сложные элементы интерфейса, работать с данными и событиями и применять стилизацию для улучшения пользовательского опыта.

Верстка XML

Для работы с любым настольным приложением нужен хороший интерфейс. Создание интерфейса или Fronted разработка непростая задача, но, как и у любой разработки у процесса создания интерфейса существуют простые правила, соблюдая которые, можно за короткое время быстро наклепать какой никакой интерфейс. И в этой главе будут разобраны основные правила разработки интерфейса, а также UI-элементы WPF.

База

Для верстки интерфейса в WPF используется XML.

XML — это язык разметки подобный HTML. Расшифровывается как (англ. Extensible Markup Language - Расширяемый Язык Разметки). В отличии от остальных языков разметки, XML сам по себе не определён (это означает, что вы должны сами определять используемые теги). Основной целью XML является передача данных между разными системами (даже концептуально разными), такими как интернет.

Верстка пишется в файлах с расширением .xml (Например MainWindow.xml). Примерная Структура Верстки:

```
<Window "куча различных свойств">
  <Grid>
      <НазваниеЭлемента "какие-то свойства" />
  </Grid>
</Window>
```

В XML каждый элемент можно объявить двумя способами:

```
1.  <НазваниеЭлемента />
2.  <НазваниеЭлемента ></НазваниеЭлемента>
```

В первом случае название элемента заключается в теги < /> и элемент считается закрытым. Данное объявление элемента рекомендуется использовать для таких элементов, которые внутри себя ничего не размещают: поля ввода (TextBox), выпадающие списки (ComboBox), надписи (TextBlock) и другие простые элементы, так как данная форма занимает визуально меньше места в вашей верстке. По факту всегда можно использовать вторую, ошибкой считаться не будет.

Вторая форма подходит для объявления, таких элементов, которые внутри себя размещают другие элементы между тегами **> <**. Ее вы можете увидеть у элемента Grid или Window в примере выше. Рекомендуется использовать ее при объявлении контейнеров типа Grid, StackPanel, WrapPanel, DockPanel и других сложных свойств, которые требуют наличия внутри себя других элементов.

У каждого элемента можно настраивать большое количество свойств. Свойства отделяются друг от друга пробелами и вписываются после имени элемента и до тега /> или > в зависимости от способа объявления.

```
<НазваниеЭлемента Свойство="Значение"
                  Свойство="Значение" />

<НазваниеЭлемента Свойство="Значение"
                  Свойство="Значение">
</НазваниеЭлемента>
```

Интерактивные и не очень элементы

TextBlock

TextBlock — это элемент, который представляет какую-то надпись посредством свойства Text.

```
<TextBlock Text="Hello world!" />
```

Основные свойства TextBlock’ а:

- FontSize – размер шрифта
- FontWeight – толщина шрифта
- Foreground – цвет шрифта
- Foreground – цвет шрифта
- Text – отображаемый текст

TextBox

TextBox - поле ввода, сюда можно писать.

```
<TextBox />
```

Основные свойства TextBox’ а:

- FontSize – размер шрифта
- FontWeight – толщина шрифта
- Foreground – цвет шрифта
- Text – введённый текст

Button

Button – кнопка, на которую можно клацать и что-то будет происходить.

```
<Button />
```

Основные свойства Button'a:

- FontSize – размер шрифта
- FontWeight – толщина шрифта
- Foreground – цвет шрифта
- Background – цвет кнопки
- Content – подпись кнопки
- Click – событие, которое будет происходить по нажатию на кнопку

ComboBox

ComboBox – выпадающий список, содержащий определенный набор элементов для выбора.

```
<ComboBox />
```

Для того чтобы в ComboBox были элементы для выбора есть несколько способов как их добавить.

1. Добавление элементов на .xaml.

Для этого внутрь ComboBox'a необходимо добавить элементы ComboBoxItem и прописать им свойство Content, которое определяет название элемента при его выборе на интерфейсе.

```
<ComboBox>
    <ComboBoxItem Content="Вариант 1" />
    <ComboBoxItem Content="Вариант 2" />
    <ComboBoxItem Content="Вариант 3" />
</ComboBox>
```

2. Использование свойства Items в .cs

Данный способ похож на предыдущий только ComboBox'у необходимо задать Name и обратиться к нему в коде, к свойству Items и вызвать метод Add(), который принимает строковое значение.

```
// Конструктор главного окна
public MainWindow()
{
    InitializeComponent();
    // CBVariants это название,
    // которое указывается в x:Name
    CBVariants.Items.Add("Вариант 1");
    CBVariants.Items.Add("Вариант 2");
    CBVariants.Items.Add("Вариант 3");
}
```

3. Использование ItemsSource

Данный способ предполагает, что для ComboBox'а уже имеется готовый список значений и его можно полностью назначить как источник для элементов. Обычно это список объектов определенного класса, который представляет собой перечисление (например: Роль – администратор, клиент, пользователь и т.п.).

Чаще всего данный способ применяется, когда ComboBox заполняется данными из БД.

```
public MainWindow()
{
    InitializeComponent();
    CBVariants.ItemsSource = new List<Role>() { new Role() { Name = "Администратор"},
                                                new Role() { Name = "Клиент" },
                                                new Role() { Name = "Пользователь" } };
}
```

Однако при использовании данного способа также потребуется прописать свойство DisplayMemberPath в .xaml. Это необходимо для того, чтобы ComboBox понимал какое свойство переданного класса ему отображать на интерфейсе.

```
<ComboBox x:Name="CBVariants"
          DisplayMemberPath="Name" />
```

Основные свойства ComboBox'а:

- DisplayMemberPath – определение свойства класса для отображения
- SelectionChanged – событие изменения выбора элемента
- ItemsSource – источник элементов
- Items – коллекция текущих элементов
- SelectedItem – выбранный элемент
- SelectedIndex – индекс выбранного элемента в коллекции

DatePicker

DatePicker –элемент для выбора даты. Часто форма требует выбора даты пользователем и использовать для этого обычный TextBox бывает неудобно ввиду валидации формата введенной даты или тяжелой логики ограничения выбора определенных дат.

```
<DatePicker />
```

Основные свойства DatePicker’a:

- SelectedDate – выбранная дата
- SelectedDateChanged – событие изменения выбранной даты
- BlackoutDates – недопустимые для выбора даты. Набор дат, которые невозможно выбрать через интерфейс задается только через код.

```
// В DatePicker можно выбрать только текущию и будущие даты
DPDate.BlackoutDates.AddDatesInPast();
// Добавление определенных дат
DPDate.BlackoutDates.Add(new CalendarDateRange(new DateTime(2025, 1, 1)));
DPDate.BlackoutDates.Add(new CalendarDateRange(new DateTime(2025, 2, 1)));
```

Контейнеры

Любой элемент будь то TextBlock или Button должен находится контейнере.

Контейнеры — это такие элементы, основная задача которых это содержать внутри себя другие элементы и размещать их определённым образом.

Grid

Grid — это контейнер, который можно разбивать на колонки и размещать внутри него элементы, указывая индекс (0 – это первый элемент, 1 – второй элемент и т.д.) колонки или строки.

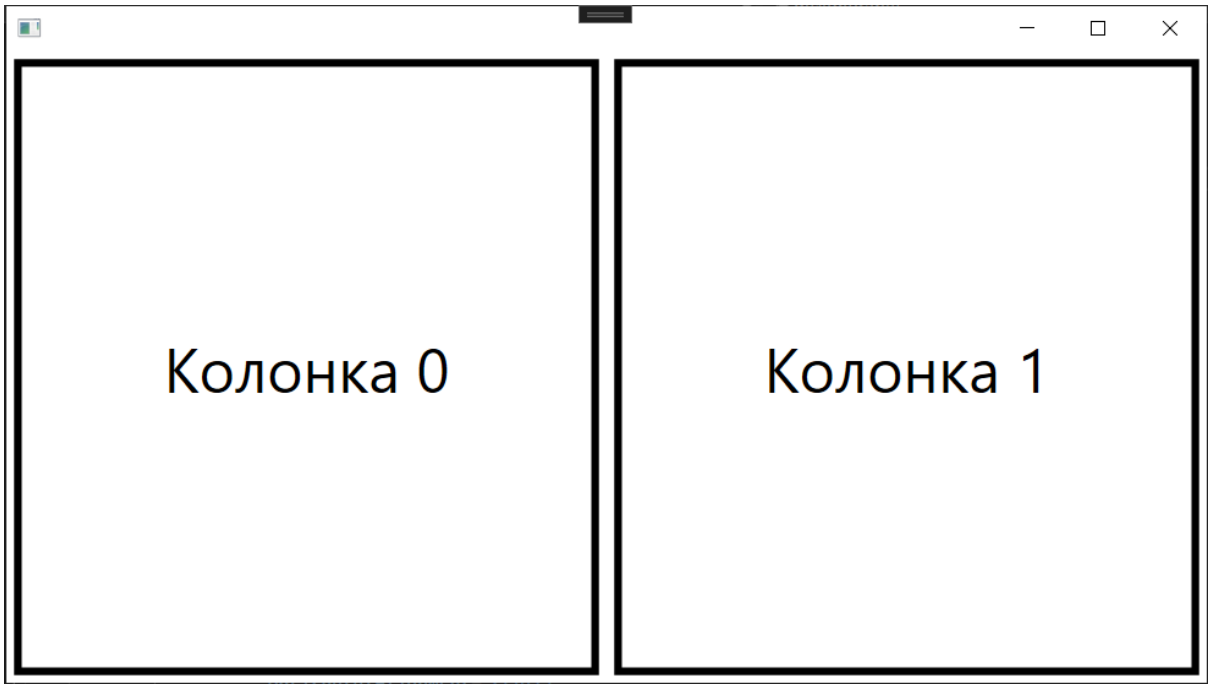
Как правило Grid является главным родительским элементом на окне, так как он позволяет удобно разбить интерфейс на области.

```
<Grid>
    “Какие-то элементы”
</Grid>
```

Для того чтобы настроить колонки необходимо прописать сложное свойство Grid.ColumnDefinitions. Свойство Grid.ColumnDefinitions принимает в себя элементы ColumnDefinition. Каждый ColumnDefinition отвечает за одну колонку. По умолчанию у Grid'а одна колонка.

Ниже показан Grid, у которого определено две колонки.

```
<Grid>
    <Grid.ColumnDefinitions>
        <ColumnDefinition />
        <ColumnDefinition />
    </Grid.ColumnDefinitions>
</Grid>
```



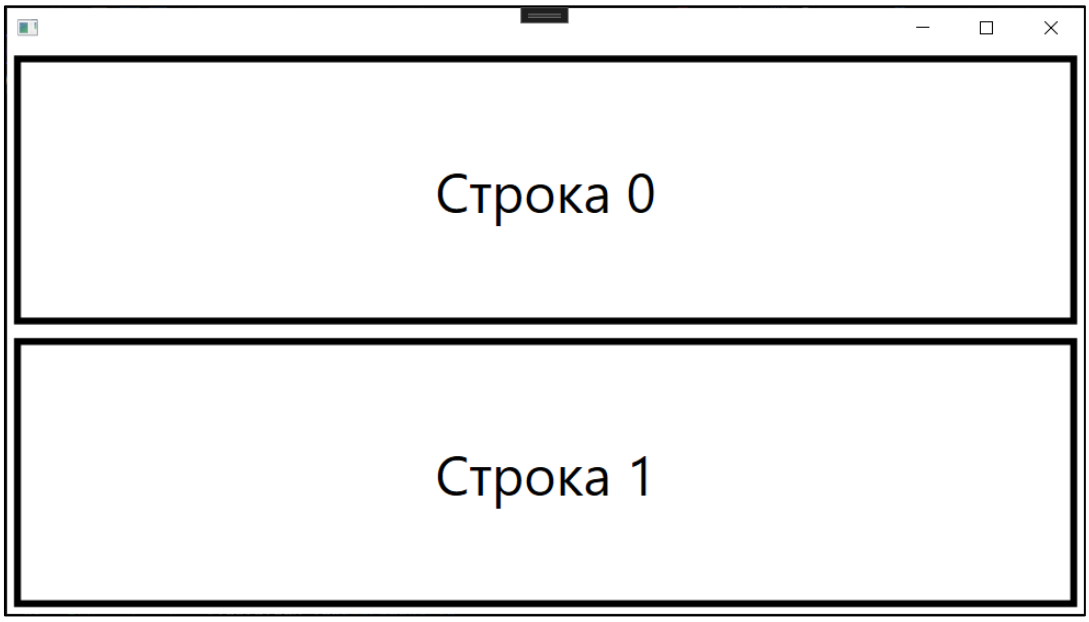
Ширину колонок можно задавать через свойство Width нужного ColumnDefinition.

```
<Grid>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="150" />
        <ColumnDefinition />
    </Grid.ColumnDefinitions>
</Grid>
```

Для того чтобы настроить строки необходимо прописать сложное свойство Grid.RowDefinitions. Свойство Grid.RowDefinitions принимает в себя элементы RowDefinition. Каждый RowDefinition отвечает за одну строку. По умолчанию у Grid’а одна строка.

Ниже показан Grid, у которого определено две строки.

```
<Grid>
    <Grid.RowDefinitions>
      <RowDefinition />
      <RowDefinition />
    </Grid.RowDefinitions>
</Grid>
```



Высоту строк можно задавать через свойство Height нужного RowDefinition.

```
<Grid>
    <Grid.RowDefinitions>
      <RowDefinition Height="150" />
      <RowDefinition />
    </Grid.RowDefinitions>
</Grid>
```

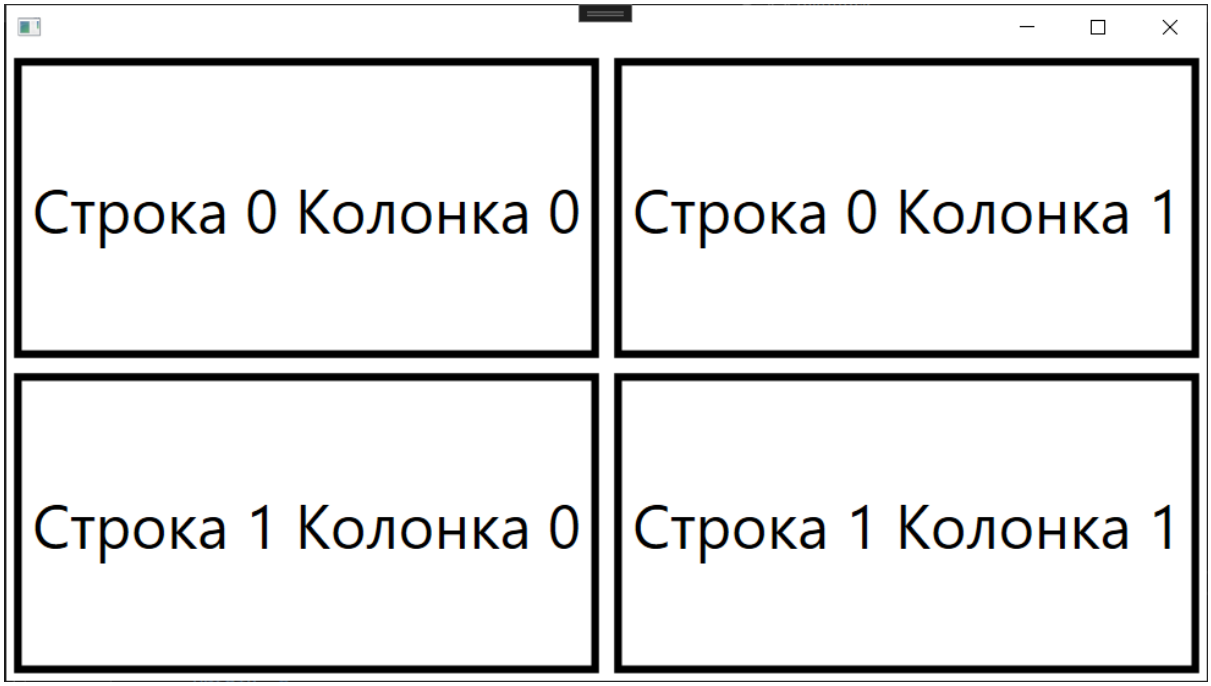
Также эти два свойства можно комбинировать, настраивая ваш Grid как сетку, которая будет состоять из ячеек.

```
<Grid>
  <Grid.RowDefinitions>
    <RowDefinition />
    <RowDefinition />
  </Grid.RowDefinitions>
  <Grid.ColumnDefinitions>
    <ColumnDefinition />
    <ColumnDefinition />
  </Grid.ColumnDefinitions>
</Grid>
```

Чтобы размещать элементы в нужные строки или колонки у элемента настраивается свойство Grid.Column для колонки или Grid.Row для строки, которые принимают индекс. По умолчанию у элементов значения свойств Grid.Column и Grid.Row равны 0.

```
<Grid>
  <Grid.RowDefinitions>
    <RowDefinition />
    <RowDefinition />
  </Grid.RowDefinitions>
  <Grid.ColumnDefinitions>
    <ColumnDefinition />
    <ColumnDefinition />
  </Grid.ColumnDefinitions>

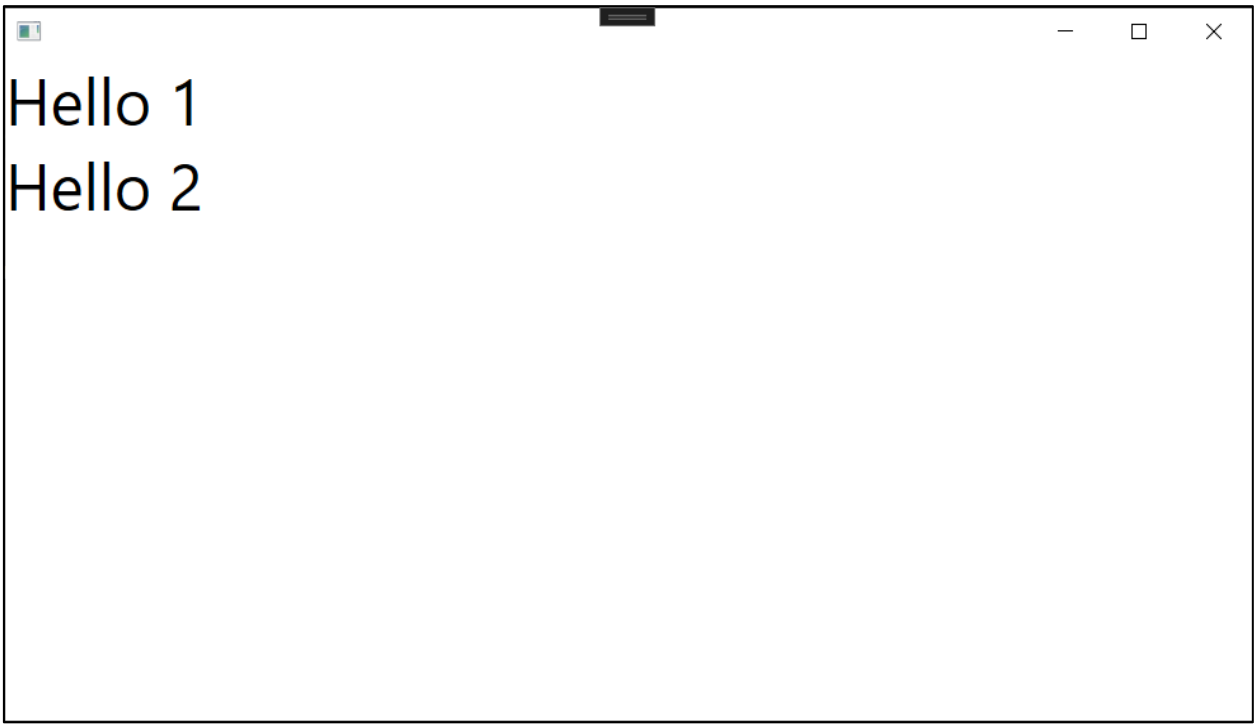
  <TextBlock Text="Строка 0 Колонка 0" />
  <TextBlock Text="Строка 1 Колонка 0"
             Grid.Row="1" />
  <TextBlock Text="Строка 0 Колонка 1"
             Grid.Column="1" />
  <TextBlock Text="Строка 0 Колонка 1"
             Grid.Row="1"
             Grid.Column="1" />
</Grid>
```



StackPanel

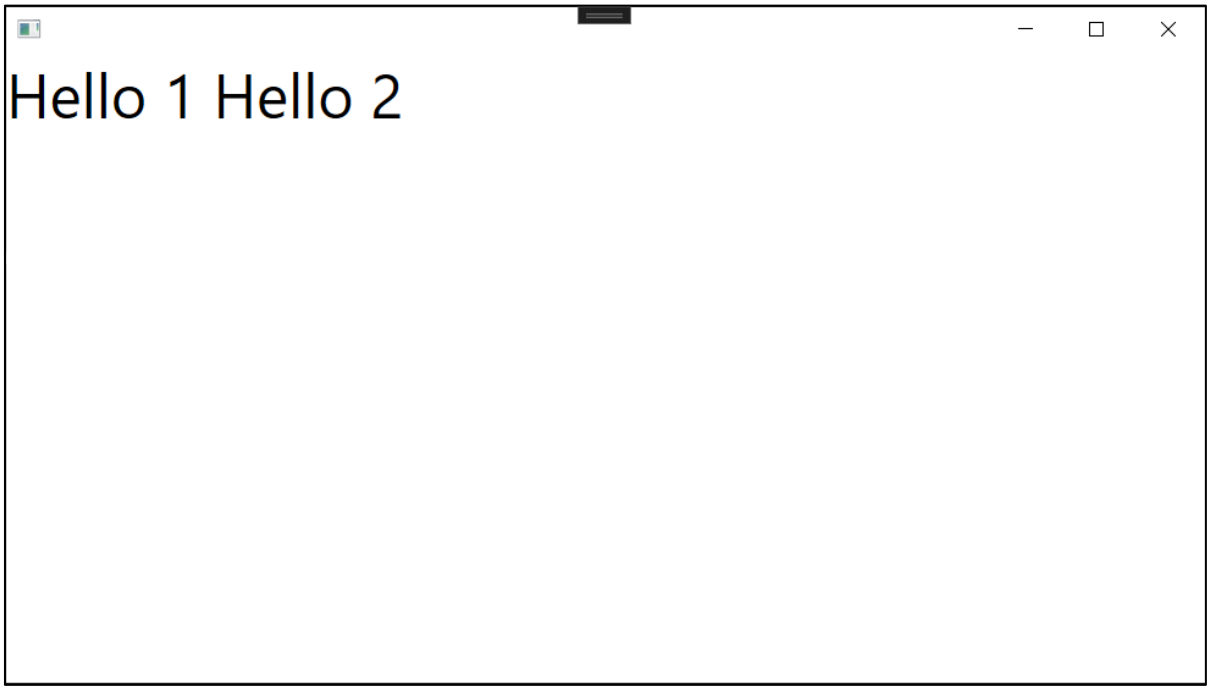
StackPanel размещает элементы внутри себя друг за другом. По умолчанию они располагаются сверху вниз, вертикально.

```
<StackPanel>
  <TextBlock Text="Hello 1" />
  <TextBlock Text="Hello 2" />
</StackPanel>
```



Чтобы размещать элементы слева направо надо задать ему свойство Orientation=”Horizontal”.

```
<StackPanel Orientation="Horizontal">
  <TextBlock Text="Hello 1" />
  <TextBlock Text="Hello 2" />
</StackPanel>
```



DockPanel

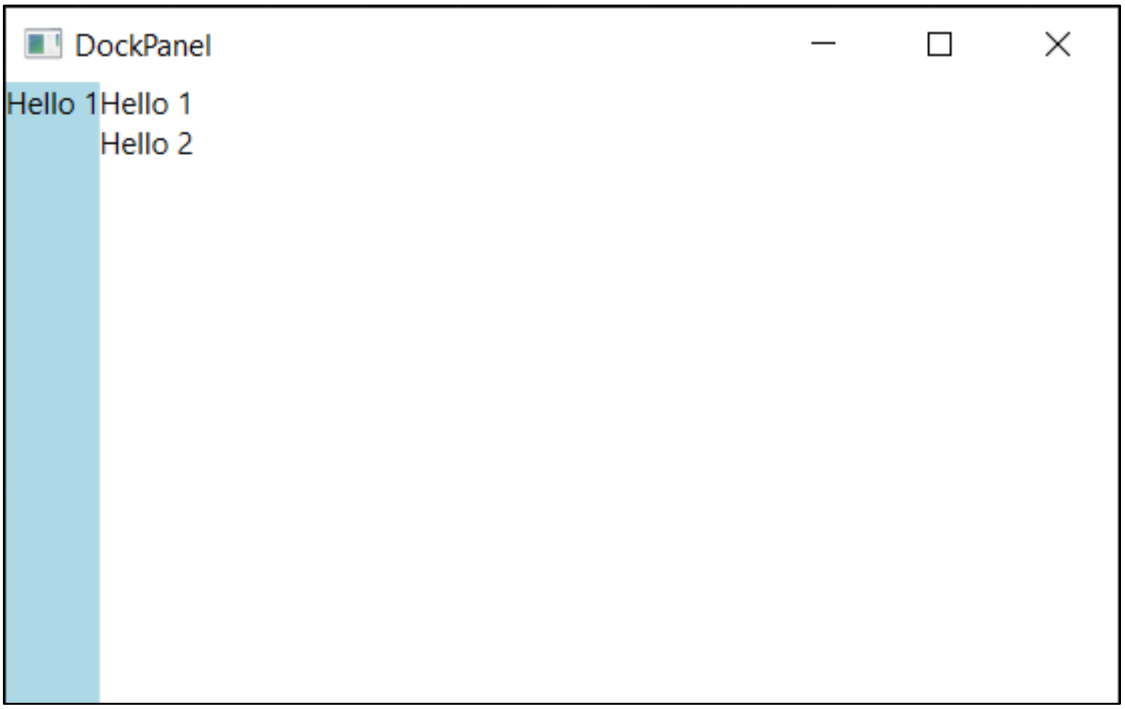
DockPanel похож на Grid, но отличается тем, что имеет предопределенные зоны вместо строк и колонок.

У DockPanel есть 5 зон: левая, правая, верхняя и нижняя (Left, Right, Top, Bottom соответственно). 5 зона расположена в центре и не требует указания.

Для того чтобы настраивать в какой зоне элементы будут расположены элементам следует указывать свойство DockPanel.Dock.

При таком свойство TextBlock всегда будет расположен слева, а элементы без свойства DockPanel.Dock будут расположены в центре.

```
<DockPanel>
    <TextBlock Text="Hello 1"
               Background="LightBlue" />
    <StackPanel>
        <TextBlock Text="Hello 1" />
        <TextBlock Text="Hello 2" />
    </StackPanel>
</DockPanel>
```

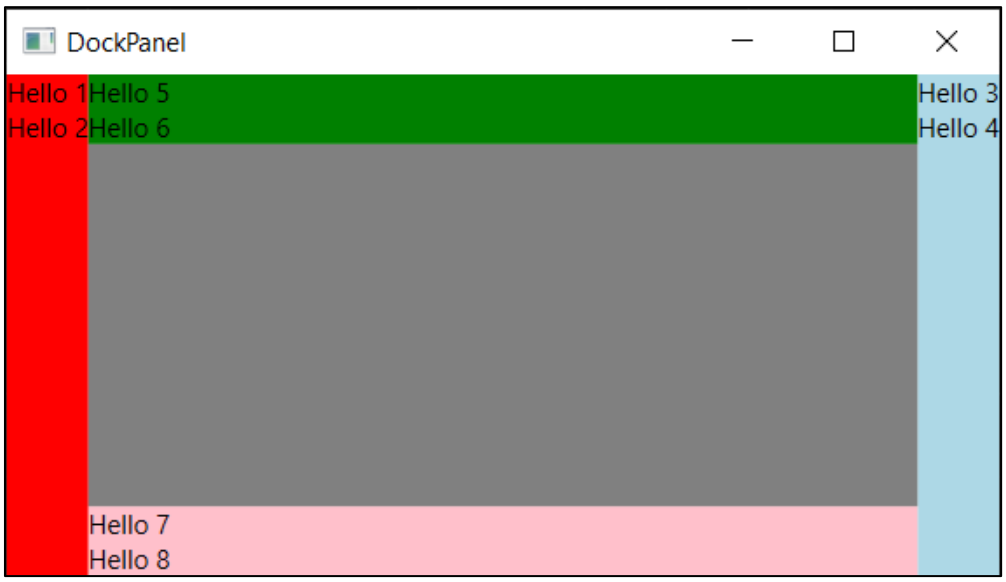


Для того чтобы расположить элемент в DockPanel по середине, этот элемент необходимо объявить ниже остальных элементов, которые имеют свойство DockPanel.Dock.

```
<DockPanel>
  <StackPanel DockPanel.Dock="Left"
              Background="Red">
    <TextBlock Text="Hello 1"/>
    <TextBlock Text="Hello 2"/>
  </StackPanel>
  <StackPanel DockPanel.Dock="Right"
              Background="LightBlue">
    <TextBlock Text="Hello 3"/>
    <TextBlock Text="Hello 4"/>
  </StackPanel>
  <StackPanel DockPanel.Dock="Top"
              Background="Green">
    <TextBlock Text="Hello 5"/>
    <TextBlock Text="Hello 6"/>
  </StackPanel>
  <StackPanel DockPanel.Dock="Bottom"
              Background="Pink">
    <TextBlock Text="Hello 7"/>
    <TextBlock Text="Hello 8"/>
  </StackPanel>
  <StackPanel Background="Gray">

  </StackPanel>
</DockPanel>
```

Элемент, который не имеет свойства DockPanel будет растягиваться по всей свободной зоне в центре.



WrapPanel

WrapPanel – контейнер который схож по принципу расположения элементов со StackPanel. По умолчанию WrapPanel располагает элементы по горизонтали слева направо. Чтобы изменить направление размещения также используется свойство Orientation, которое принимает Horizontal или Vertical значения.

Однако WrapPanel отличается от StackPanel тем, что элементы, которые не умещаются на экране при изменении размеров окна будут перемещены на соседнюю строку или колонку в зависимости от значения Orientation.

Списки

В приложениях часто требуется вывод информации в виде списка. Источником данных для таких списков могут быть как данные из БД, так и различные статичные списки, реализованные в программе. В WPF реализованы несколько элементов которые реализуют списки.

DataGrid

DataGrid – элемент для отображения списка в виде таблицы.

По умолчанию DataGrid автоматически определяет колонки таблицы в зависимости от свойств Класса, который в него передается. За это поведение отвечает свойство AutoGenerateColumns, который принимает bool. Если установить это свойство на false, то колонки автоматически сгенерированы не будут.

Использование автоматической генерации колонок полезно использовать если не требуется определенный набор колонок, в этом случае таблица будет содержать абсолютно все свойства и, если передавать в нее тип с большим количеством свойств это может быть нечитабельным.

Для определения набора колонок, которые будут отображаться в DataGrid используется свойство DataGrid.Columns, которое принимает коллекцию элементов DataGridTextColumn, DataGridTemplateColumn и др.

Общий вид DataGrid’a

```
<DataGrid>
    <DataGrid.Columns>
        Набор колонок
    </DataGrid.Columns>
</DataGrid>
```

Каждая колонка имеет свойство Header, которое определяет название заголовка и принимает строчное значение.

DataGridTextColumn используется для отображения информации в виде текста. Для того чтобы определить какое свойство класса будет отображаться в этой колонке используется свойство Binding.

В свойство Binding в фигурных скобках прописывается значение Binding за которым идет название свойства класса, который DataGrid отображает.

```
<DataGrid>
    <DataGrid.Columns>
        <DataGridTextColumn Header="Какое-то название"
                            Binding="{Binding НазваниеСвойства}"/>
    </DataGrid.Columns>
</DataGrid>
```

Допустим в проекте реализован класс:

```
class User
{
    public string Name { get; set; }
}
```

Следовательно, в Binding следует прописать “{Binding Name}” для того, чтобы привязать DataGridTextColumn свойство Name.

DataGridTemplateColumn отображает содержимое ячейки в виде настраиваемой верстки. Иногда возникают такие случае что только текста в таблице недостаточно и может потребоваться размещения в колонки таких элементов как картинки или кнопки. Как раз для этого предназначен DataGridTemplateColumn. Для определения вида ячеек, которые находятся в этой колонке используется Свойство DataGridTemplateColumn.CellTemplate, которое принимает в себя

DataTemplate, которое уже содержит шаблон для всех ячеек данной колонки.

DataTemplate может содержать только один элемент и поэтому следует первым элементом определять любой подходящий контейнер, который в свою очередь уже может содержать множество элементов верстки.

Для того чтобы ячейка отображала какие-либо свойства класса следует прописывать отдельные элементы под них, как например показано ниже:

```
<DataGrid>
    <DataGrid.Columns>
        <DataGridTemplateColumn Header="Какое-то название">
            <DataGridTemplateColumn.CellTemplate>
                <DataTemplate>
                    <StackPanel>
                        <TextBlock Text="{Binding Name}" />
                        <TextBlock Text="{Binding Surname}" />
                    </StackPanel>
                </DataTemplate>
            </DataGridTemplateColumn.CellTemplate>
        </DataGridTemplateColumn>
    </DataGrid.Columns>
</DataGrid>
```

Это два основных типа колонок, которые чаще всего используются в DataGrid.

ListView (требует доработки)

ListView является базовым элементом управления (контролом), наследующимся от класса ItemsControl, и предназначен для отображения коллекций данных. Его основная роль — служить гибким контейнером для представления объектов в произвольном, настраиваемом виде, таком как галереи, списки сообщений или плитки товаров.

В отличие от DataGrid, который накладывает строгое требование к представлению данных в виде рядов и колонок, ListView делегирует управление внешним видом и компоновкой элементам-шаблонам. Это позволяет добиться полной визуальной свободы и создать сложные элементы интерфейса, где каждый объект коллекции превращается в полноценный, настраиваемый блок.

Если DataGrid идеально подходит для редактирования и анализа табличных данных, то ListView — незаменим для высококастомизированных и визуально насыщенных списков.

Для того чтобы элемент коллекции был корректно отображен, ListView требует указания шаблона данных, который содержится в свойстве ItemTemplate. Данное свойство принимает объект DataTemplate, который является описанием XAML-разметки для одного объекта коллекции.

Внутри DataTemplate контекст данных (DataContext) автоматически устанавливается на текущий элемент, который отображается в списке. Это позволяет производить прямую привязку свойств внутренних контролов (таких как Image или TextBlock) к свойствам исходного объекта:

```
<ListView>
  <ListView.ItemTemplate>
      <DataTemplate>
          <Border>
            <StackPanel>
              <Image Source="{Binding Путь}"/>
              <TextBlock Text="{Binding НазваниеСвойства}"/>
              <Button Content="Подробнее"/>
             </StackPanel>
          </Border>
      </DataTemplate>
  </ListView.ItemTemplate>
</ListView>
```

ItemTemplate отвечает за внешний вид и содержимое отдельного элемента.

В то время как ItemTemplate определяет, как выглядит отдельный элемент, свойство ItemsPanel определяет, как эти элементы располагаются друг относительно друга внутри контейнера ListView. Это свойство принимает объект ItemsPanelTemplate, который должен содержать один из доступных контейнеров WPF.

Для создания плиточного вида с переносом строк (например, по две карточки в ряд) необходимо установить WrapPanel в ItemsPanelTemplate:

```
<ListView>
  <ListView.ItemsPanel>
    <ItemsPanelTemplate>
        <WrapPanel Orientation="Horizontal" HorizontalAlignment="Center"/>
    </ItemsPanelTemplate>
  </ListView.ItemsPanel>
</ListView>
```

При использовании WrapPanel для создания плиток, ListView по умолчанию может попытаться использовать горизонтальную прокрутку, если ширина родительского элемента не ограничена. Это приводит к тому, что элементы не переносятся, а продолжают уходить за пределы видимости.

Чтобы принудительно заставить ListView прокручиваться вертикально для отображения новых строк, необходимо явно отключить горизонтальную прокрутку, используя присоединенное свойство ScrollViewer:

```
<ListView ScrollViewer.HorizontalScrollBarVisibility="Disabled">

</ListView>
```

Это свойство является критически важным для корректной работы плиточного интерфейса, гарантируя, что элементы, перенесенные WrapPanel на новую строку, становятся доступными через вертикальную полосу прокрутки.

Навигация страницами

Навигация страницами позволяет приложению переключать активное содержимое, без необходимости отображения нескольких окон.

1. Создать папку Pages
2. Добавить в папку Pages элементы Page (страница)
3. Заполнить страницу содержимым
4. На MainWindow.xml добавить элемент Frame (это область, в которой будут отображаться страницы).

```
<Frame x:Name="MainFrame"
       NavigationUIVisibility="Hidden" />
```

Для переключения страниц внутри Frame'а в коде необходимо обратиться к Frame по его имени и вызвать метод Navigate, метод как параметр принимает объект страницы.

```
MainFrame.Navigate(new[НазваниеСтраницы]());
```

Для того чтобы при запуске приложения сразу отображалась какая-то страница, метод Navigate можно вызвать в конструкторе:

```
public MainWindow()
{
    InitializeComponent();
    MainFrame.Navigate(new[НазваниеСтраницы]());
}
```

Навигация внутри страницы

Рассмотрим такую ситуацию: на MainWindow была совершена навигация на Page1 и допустим на этой странице есть кнопка, которая переходит на следующую страницу. Мы имеем метод на странице который выполняют навигацию, но из этой страницы нет доступа к MainFrame. В этом случае вместо MainFrame следует обращаться к NavigationService.

```
NavigationService.Navigate(new[НазваниеСтраницы]());
```

Обратная навигация

Также постраничная навигация дает возможность возвращаться на предыдущие страницы, для этого используется метод GoBack().

```
//Если имеется доступ к Frame
MainFrame.GoBack();
//Если это внутри страницы
NavigationService.GoBack();
```

Подключение БД

Создание модели

1. Создать папку Models в корне проекта

2. Добавить элемент Модель ADO.NET EDM из вкладки “Данные” и назначить ему название (по дефолту можно назвать также как и называется БД на сервере)

3. В открывшемся окне выбрать Конструктор EF из базы данных, это позволит создать структуру классов на основе БД

4. Далее необходимо создать соединение

5. Необходимо заполнить имя сервера, его можно взять из MSSQL, нажав правой кнопкой по серверу слева и выбрать свойства в контекстном меню

6. В выпадающем списке ниже необходимо выбрать базу данных

7. На окне выбора объектов необходимо выбрать галочку “Таблицы” и убрать галочку “Формировать имена объектов во множественном или единственном числе”

8. Готово

Создание единой переменной для связи с БД

1. Открыть файл App.xml.cs (нажать на стрелочку рядом с App.xml)

2. Создать переменную DB

```
public partial class App : Application
{
    public static [НазваниеМодели] Entities DB = new [НазваниеМодели] Entities();
}
```

На этом этапе может возникнуть проблема, какое имя модели. Имя модели видно справа в обозревателе решений и к этому имени модели надо приписать Entities. Также надо подключить using для того, чтобы можно было обращаться к модели.

```
using [НазваниеПроекта].[НазваниеПапкиГдеЛежитМодель];

public partial class App : Application
{
    public static [НазваниеМодели] Entities DB = new [НазваниеМодели] Entities();
}
```

К этой переменной можно обращаться из любого класса, прописав App.DB. Эта переменная позволяет обращаться через свойства к таблицам в БД, эти свойства будут называться также как и названы таблицы на сервере

```
// Получение списка из таблицы на сервере
App.DB.[НазваниеТаблицы].ToList()
```

Обновление модели

Если на сервере были какие, то изменения в структуре (добавлены таблицы, модифицированы существующие таблицы и т.п.), то модель в проекте необходимо обновить.

1. Открыть диаграмму модели, дважды нажав на модель

2. Выделить все таблицы на диаграмме и удалить их

3. Правой кнопкой открыть контекстное меню и выбрать пункт обновить модель из базы данных. 4. Нажать на галочку Таблицы

5. Готово

6. Сохранить изменения (ctrl+shift+s)

Добавление данных в БД

Для добавления новой записи в БД используется следующая конструкция:

```
App.DB.НазваниеТаблицы.Add(объект);
App.DB.SaveChanges();
```

App.DB — это переменная для подключения БД (см. Подключение БД).

Объект — это объект класса, который создал EntityFramework на основе таблицы из БД.

SaveChanges — это метод для сохранения изменений.

Алгоритм для реализации формы с добавлением данных в БД

1. Реализовать страницу, на которой будет происходит заполнение записи

2. В коде подключить using к папке, где находится модель, чтобы пользоваться классами модели

3. В Click'е кнопки, которая отвечает за добавление записи прописать следующий код

```
// создание объекта который необходимо заполнить данными
НазваниеТаблицы названиеОбъекта = new НазваниеТаблицы();
// заполнение Атрибута1 объекта строкой из TextBox
названиеОбъекта.Атрибут1 = TBАтрибут1.Text;
// заполнение остальных атрибутов аналогичным способом
App.DB.НазваниеТаблицы.Add(названиеОбъекта);
App.DB.SaveChanges();
```

Следует учитывать, что при выполнении метода Add передаваемый объект должен быть валидным для БД, то есть все атрибуты должны быть заполнены подходящими по типу данными, обязательные атрибуты также должны быть заполнены. В противном случае метод Add выполнится вызовет исключение (см. Валидация свойств объекта).

Реализация формы через Binding

Работа формы через Binding (привязка) позволяет привязать объекты модели к форме, избегая ручного заполнения его свойств.

Рассмотрим реализацию формы через Binding на примере добавление записи в таблицу Pizza (Id - int, Name - nvarchar (50), Cost - float, Diameter - float)

1. Создать страницу PizzaPage

2. Изменить конструктор странницы, прописав параметр Pizza pizza. Данная модификация конструктора страницы позволит передавать объект, с которым страница должна работать

```
public PizzaPage(Pizza pizza)
{
    InitializeComponent();
}
```

3. Добавить на страницу переменную Pizza contextPizza для того, чтобы обращаться к переданному объекту в любой части страницы

```
Pizza contextPizza;
public PizzaPage(Pizza pizza)
{
    InitializeComponent();
}
```

4. Прописать в конструктор следующий код:

```
Pizza contextPizza;
public PizzaPage(Pizza pizza)
{
    InitializeComponent();
    contextPizza = pizza;
    DataContext = contextPizza;
}
```

Данная конструкция связывает свойства передаваемого объекта и элементы ввода.

5. Заполнить PizzaPage.xml верстkой для заполнения свойств пиццы

```
<Grid>
  <StackPanel>
    <TextBlock Text="Название" />
    <TextBox />
    <TextBlock Text="Стоимость" />
    <TextBox />
    <TextBlock Text="Диаметр" />
    <TextBox />
    <Button x:Name="BSave"
            Content="Сохранить"
            Click="BSave_Click" />
    <Button x:Name="BCancel"
            Content="Отмена"
            Click="BCancel_Click" />
  </StackPanel>
</Grid>
```

6. Создать привязки к полям ввода. Это позволяет понять элементам, к каким свойствам объекта (который находится в DataContext) привязаться

```
<Grid>
  <StackPanel>
    <TextBlock Text="Название" />
    <TextBox Text="{Binding Name}" />
    <TextBlock Text="Стоимость" />
    <TextBox Text="{Binding Cost}" />
    <TextBlock Text="Диаметр" />
    <TextBox Text="{Binding Diameter}" />
    <Button x:Name="BSave"
            Content="Сохранить"
            Click="BSave_Click" />
    <Button x:Name="BCancel"
            Content="Отмена"
            Click="BCancel_Click" />
  </StackPanel>
</Grid>
```

7. На клике BSave_Click в PizzaPage.xaml.cs прописать код для сохранения модели. Проверка if (contextPizza.Id == 0) позволяет понять добавляется новая пицца в БД или редактируется существующая. Так как у новых объектов модели Id по умолчанию устанавливается 0 — это дает понять, что данный объект необходимо добавить, в ином случае если у объекта уже есть Id в БД, то его необходимо только сохранить, изменив значения его свойство

```
private void BSave_Click(object sender, RoutedEventArgs e)
{
    if (contextPizza.Id == 0)
        App.DB.Pizza.Add(contextPizza);
    App.DB.SaveChanges();
    // Переход назад после сохранения изменений
    NavigationService.GoBack();
}
```

8. На форме с общим списком реализовать две кнопки: Добавление пиццы, Редактирование Пиццы

```
<Button x:Name="BAddPizza"
        Content="Добавление пиццы"
        Click="BAddPizza_Click" />
<Button x:Name="BEditPizza"
        Content="Редактирование пиццы"
        Click="BEditPizza_Click" />
```

9. Прописать клики для кнопок. Так как конструктор страницы PizzaPage в параметре всегда требует объект Pizza, он должен быть передан при навигации

В случае если происходит добавление новой записи, то в конструктор передается новый объект (new Pizza()).

В случае если происходит редактирование выбранного объекта из списка, его сначала необходимо получить. Для получения выделенного объекта необходимо обратиться к списку (DGPizza) и обратиться к свойству SelectedItem. Значение SelectedItem необходимо преобразовать оператором as к нужному типу (в данном случае Pizza). В случае если selectedPizza == null это означает что пользователь перед тем, как отредактировать не выбрал запись. Далее selectedPizza передается в параметр конструктора страницы (new PizzaPage(selectedPizza)).

```
private void BAddPizza_Click(object sender, RoutedEventArgs e)
{
    NavigationService.Navigate(new PizzaPage(new Pizza()));
}

private void BEditPizza_Click(object sender, RoutedEventArgs e)
{
    var selectedPizza = DGPizza.SelectedItem as Pizza;
    if (selectedPizza == null)
    {
        MessageBox.Show("Выберите пиццу!");
        return;
    }
    NavigationService.Navigate(new PizzaPage(selectedPizza));
}
```

Данный метод позволяет быстро реализовать одновременно два функционала – добавление и редактирование, избегая ручного заполнения всех свойств в коде.

Валидация свойств объекта

Перед добавлением/редактированием записи в БД объект должен пройти валидацию, проверку всех свойств на корректные значения. Должны совпадать типы данных, длина значений и возможно какие-то пользовательские проверки, которые требуются по предметной области.

Реализация валидации

В Click, который отвечает за отправку записи в БД прописать следующий код:

```
string errorMessage = "";
// Различные проверки на валидность
if (!string.IsNullOrWhiteSpace(errorMessage))
{
  MessageBox.Show(errorMessage); return;
}
// Добавление и сохранение в бд
```

Переменная errorMessage хранит все сообщения об ошибках, которые совершил пользователь на форме. Изначально эта переменная пустая, по мере того как будут выполняться проверки на валидность эта переменная будет заполняться различными сообщениями. После того как все проверки были выполнены переменная errorMessage проверяется на пустоту статическим методом класса string IsNullOrWhiteSpace который принимает строку и возвращает bool. В случае если переданная в метод строка не пустая, а значит имеет какие-то сообщения об ошибках в валидности объекта, необходимо отобразить текст с ошибками через MessageBox и выполнить return чтобы завершить выполнение метода добавления в БД.

Пример:

```
string errorMessage = "";
if(string.IsNullOrWhiteSpace(context.Свойство))
  errorMessage += "Заполните свойство\n";

if(context.Свойство == 0)
  errorMessage += "Свойство не может быть 0\n";

if(context.Свойство > 100)
  errorMessage += "Свойство должно быть меньше 100\n";

if (!string.IsNullOrWhiteSpace(errorMessage))
{
  MessageBox.Show(errorMessage); return;
}

if(context.Id == 0)
  App.DB.Таблица.Add(context);
App.DB.SaveChanges();
```

Проверки на валидность

Проверка на пустоту строкового свойства

```
if(string.IsNullOrWhiteSpace(context.Свойство))
  errorMessage += "Заполните свойство\n";
```

Проверка на длину строкового свойства

```
if(context.Свойство.Length > максимальнаяДлина)
  errorMessage += "Свойство должно быть менее максимальнаяДлина\n";
```

Проверка числового свойства

В случае использования Binding числовые свойства привязанные к TextBox'у при некорректности значения будут 0. Также можно выполнять проверки на максимальные значения и т.д.

```
if(context.Свойство == 0)
  errorMessage += "Свойство не может быть 0\n";

if(context.Свойство > 100)
  errorMessage += "Свойство должно быть меньше 100\n";

if(context.Свойство > 100 || context.Свойство < 50)
  errorMessage += "Свойство должно быть в диапазоне 50 - 100\n";
```

Проверка на свойства связанных таблиц

В случае если объект имеет связь с другой таблицей, например User Role модель будет иметь свойство Role и как правило такие свойство заполняются через ComboBox, и привязка идет именно на свойство Role. Так как значение свойства Role является объектом класса Role то если значение не было выбрано то и значение свойства будет равно null.

```
if(context.Свойство == null)
  errorMessage += "Выберите свойство\n";
```

Проверка даты

```
if(context.ДатаРождения.Date == DateTime.Now.Date)
  errorMessage += "День рождения не может быть сегодняшним днем\n";

if(context.ДатаСозданияЗаказа > DateTime.Now)
  errorMessage += "Дата заказа должна быть меньше или равно сегодняшнего дня\n";

if(context.ДатаПериодаОт > ДатаПериодаДо)
  errorMessage += "ДатаПериодаОт должна быть меньше или равна ДатаПериодаДо\n";
```

Проверка через регулярные выражения (скоро)

LINQ и лямбда выражения (скоро)

LINQ

LINQ (Language-Integrated Query) представляет простой и удобный язык запросов к источнику данных. В качестве источника данных может выступать объект, реализующий интерфейс IEnumerable (например, стандартные коллекции, массивы, таблицы созданные EntityFramework). Но вне зависимости от типа источника LINQ позволяет применить ко всем один и тот же подход для выборки данных.

Существует несколько разновидностей LINQ:

- LINQ to Objects - применяется для работы с массивами и коллекциями такими как List, Array, Dictionary и др.;
- LINQ to Entities - используется при обращении к базам данных через технологию Entity Framework.

Помимо LINQ to Objects и LINQ to Entities существуют LINQ to XML, to DataSet, Parallel LINQ.

LINQ to Objects в своей реализации использует алгоритмы на C#. Поэтому данный тип LINQ поддерживает все возможности C#.

В отличии от LINQ to Objects, LINQ to Entities сначала преобразует LINQ запросы в SQL код, который позже можно отправить на сервер БД для его обработки и возврата результата запроса в виде подготовленных для C# кода данных.

Например, данный запрос, который запрашивает все записи из таблицы User в Базе данных сначала преобразуется в SQL код, который не выполнится пока в программе этот запрос не вызвать.

```
var usersQuery = App.DB.User.Where(x => x.Name == "Jane Doe");
```

На данный момент в переменной usersQuery будет находится следующий SQL код готовый к отправке на сервер.

```
SELECT * FROM User
WHERE User.Name = 'Jane Doe'
```

Отправить данный запрос на сервер чтобы получить необходимые записи из БД можно вызвав метод ToList().

```
var usersQuery = App.DB.User.Where(x => x.Name == "Jane Doe");
var users = usersQuery.ToList();

// либо можно упростить запись
var users = App.DB.User.Where(x => x.Name == "Jane Doe").ToList();
```

Из этого следует, что лямбда-выражение может принимать только те операции, которые могут быть интерпретированы в SQL код. Например, логическое сравнение двух значений легко записать в SQL. А вот, например использование методов, которые вы реализовали у себя в программе не смогут быть прописаны в SQL и из-за этого может возникать ошибка.

Для того чтобы использовать методы, которых нет в SQL можно сначала получить данные без его использования (например, после выполнения метода ToList(), а уже потом использовать методы, которые есть только в C#.

Лямбда-выражения

Лямбда-выражение – это анонимные методы (функции). Они используются для краткости и гибкости, позволяя определять небольшие блоки кода непосредственно там, где они нужны, без создания полноценного метода.

В примере ниже реализован метод Check с входным параметром User и проверяет логин пользователя. Аналогично можно записать этот метод в одну строку используя лямбда выражение.

```
bool Check(User x)
{
    if (x.Login == "qwerty")
        return true;
    else
        return false;
}

x => x.Login == "qwerty"
```

При составлении выражений следует указывать необходимые параметры и учитывать какой возвращаемый тип будет в результате выполнения выражения.

В общем виде лямбда-выражение имеет форму:

```
параметр => код
(параметр1, параметр2) => код
параметр => {код}
```

Параметры можно называть, как угодно, а их типы определяются методами, в которых эти лямбда-выражения применяются (например, если писать лямбда-выражение для метода FirstOrDefault, то необходимо объявить один параметр тип, которого будет определяться типом коллекции, на которой метод FirstOrDefault вызывается).

```
List<int> ints = new List<int>();
// в данном случае параметр лямбда-выражения имеет тип int
// так как метод FirstOrDefault вызывается на коллекции типа int
int firstValue = ints.FirstOrDefault(x => x > 0);

List<User> users = new List<User>();
// в данном случае параметр лямбда-выражения имеет тип User
// так как метод FirstOrDefault вызывается на коллекции типа User
User firstUser = users.FirstOrDefault(x => x.Name == "Jane Doe");
```

В примере выше при вызове FirstOrDefault параметр x имеет тип User, User – это класс, в котором реализовано свойство Name. Для того чтобы обращаться к свойствам классе если параметр лямбда-выражения имеет тип класса, следует обращаться через точку: параметр.НазваниеСвойства.

Основные методы LINQ

Дисклеймер. в приведенных ниже примерах использования LINQ методов будет использоваться следующая модель:

```
public class Pokemon
{
    public int Id { get; set; } // идентификатор
    public string Name { get; set; } // имя
    public string Type { get; set; } // тип (стихия)
    public double Weight { get; set; } // вес
    public double Height { get; set; } // рост
    public List<string> Abilities { get; set; } // способности
    public int Evolution { get; set; } // степень эволюции от 1 до n
}
```

FirstOrDefault

FirstOrDefault возвращает из коллекции самую первую запись, в случае если в коллекции нет элементов, то результатом будет значение по умолчанию, например null.

Также у FirstOrDefault есть перегрузка, которая принимает лямбда выражение для задачи условия выборки первого элемента. В этом случае метод будет искать первый элемент, который соответствует условию, если подходящего элемента в коллекции нет или коллекция пустая, результатом будет значение по умолчанию, например null.

```
// получение первой записи в коллекции
App.DB.Pokemon.FirstOrDefault();
// получение первой записи у которой Name Пикачу
App.DB.Pokemon.FirstOrDefault(x => x.Name == "Пикачу");
// поиск записи по Id
App.DB.Pokemon.FirstOrDefault(x => x.Id == 101);
// поиск записи по нескольким параметрам с использованием логического И &&
App.DB.Pokemon.FirstOrDefault(x => x.Type == "Водный" && x.Evolution == 1);
// поиск по одному из нескольких параметров с использование логического ИЛИ ||
App.DB.Pokemon.FirstOrDefault(x => x.Type == "Огненный" || x.Height > 20);
// поиск записи у которой есть свойство в виде коллекции
App.DB.Pokemon.FirstOrDefault(x => x.Abilities.Contains("Водный удар"));
```

LastOrDefault

LastOrDefault работает аналогично FirstOrDefault, но возвращает последний элемент коллекции или значение по умолчанию.

LastOrDefault также принимает лямбда-выражение для условия поиска последнего элемента.

```
// получение последней записи в коллекции
App.DB.Pokemon.LastOrDefault();
// получение последней записи у которой Height больше 25
App.DB.Pokemon.LastOrDefault(x => x.Height > 25);
```

Where

Where используется для фильтрации коллекции по условию. Возвращает метод where новую коллекцию типа IEnumerable<T>. В параметр метода передается лямбда-выражение для определения условия фильтрации.

При вызове метода where следует обращать внимание в каком контексте он вызывается LINQ to Object или LINQ to Entities (подробнее см. LINQ).

```
// получение всех записей, у которых Type Травянной
App.DB.Pokemon.Where(x => x.Type == "Травянной");
// получение всех записей по нескольким критериям
App.DB.Pokemon.Where(x => x.Height > 100 && x.Weight > 20);
// получение всех записей по одному из нескольких критериев
App.DB.Pokemon.Where(x => x.Evolution == 5 || x.Type == "Огненный");

//Комбинирование Where и FirstOrDefault
App.DB.Pokemon.Where(x => x.Name == "Чармандер").FirstOrDefault(x => x.Height > 50);
```

OrderBy/OrderByDescending/ThenBy/ThenByDescending

В LINQ также присутствует возможность сортировки коллекций.

OrderBy принимает в параметр лямбда-выражение где необходимо выбрать свойство по которому коллекция будет отсортирована. OrderBy возвращает новую коллекцию IOrderedEnumerable<T> отсортированную по возрастанию по заданному свойству.

OrderByDescending работает аналогично OrderBy, но сортировка происходит по убыванию.

```
// сортировка списка по возрастанию по свойству Type
App.DB.Pokemon.OrderBy(x => x.Type);
// сортировка списка по убыванию по свойству Height
App.DB.Pokemon.OrderBy(x => x.Height);
// OrderBy можно использовать в связке с FirstOrDefault
// это позволит получить например самого тяжелого покемона или самого легкого
// главное учитывать спецификацию предметной области
// получение самого тяжелого покемона
App.DB.Pokemon.OrderByDescending(x => x.Weight).FirstOrDefault();
```

ThenBy и ThenByDescending вызываются на уже отсортированных коллекциях когда необходимо отсортировать коллекцию по нескольким условиям или свойствам.

```
// сортировка по двум параметрам
// изначально коллекция сортируется по Name
// далее учитывая сортировку по Name выполняется сортировка по Height
// таким образом если в коллекции есть две записи с одинаковым Name, то
// они сортируются уже по Height
App.DB.Pokemon.OrderBy(x => x.Name).ThenByDescending(x => x.Height);
```

Sum

Sum позволяет подсчитать сумму всех численных свойств у объектов в коллекции. Как параметр принимает лямбда-выражение с необходимым свойством или операциями над свойством.

```
// сумма свойства Height всех записей коллекции
App.DB.Pokemon.Sum(x => x.Height);
// сумма свойства Weight поделенная на 100 всех записей коллекции
App.DB.Pokemon.Sum(x => x.Weight / 100);
// суммарный вес всех покеменов типа Огненный
App.DB.Pokemon.Where(x => x.Type == "Огненный").Sum(x => x.Height);
```

Average

Average возвращает среднее значение численного свойства, переданного в параметр как лямбда-выражение.

```
// среднее значение свойства Height
App.DB.Pokemon.Average(x => x.Height);
// среднее количество способностей у покемонов
App.DB.Pokemon.Average(x => x.Abilities.Count());
```

Count

Метод Count позволяет подсчитать количество элементов в коллекции.

Перегрузка метода принимает в параметр лямбда-выражение для выборки элементов, которые необходимо подсчитать.

```
// получение количества записей в коллекции
App.DB.Pokemon.Count();
// получение количества записей соответствующих заданному условию
App.DB.Pokemon.Count(x => x.Name == "Пикачу");
```

Min

 Скоро...

Max

 Скоро...

Select

 Скоро...

All

 Скоро...

Any

 Скоро...

GroupBy

 Скоро...

ToDictionary

 Скоро...

Стилизация интерфейса приложения (нескоро)

Разработка базы данных в MS SQL

СОВЕТЫ

1. Все таблицы обязательно должны иметь первичный ключ

2. Если вы закончили работу по созданию диаграммы или редактированию не забудьте сохранить изменения комбинацией Ctrl+Shift+S или двойной дискетой слева сверху

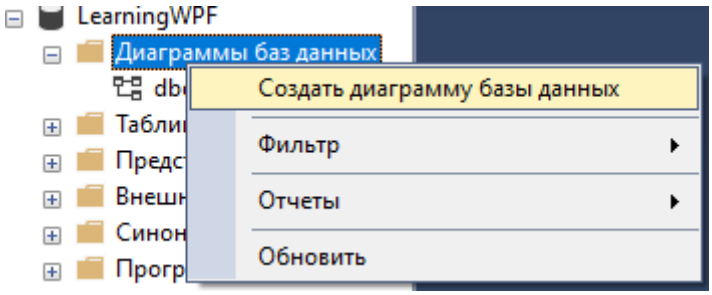


3. Если вы видите, что у названия таблицы стоит звездочка *, значит таблица отредактирована и нужно сохранить изменения

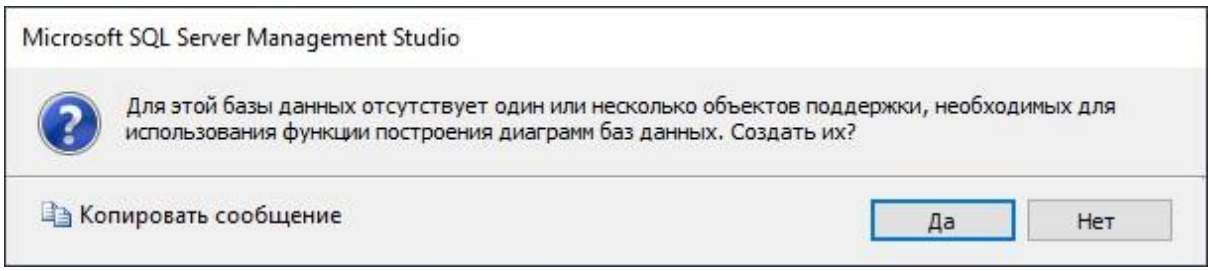
Создание диаграммы

Диаграмма в MS SQL позволяет быстро создавать новые сущности и изменять существующие, а также создавать связи между сущностями.

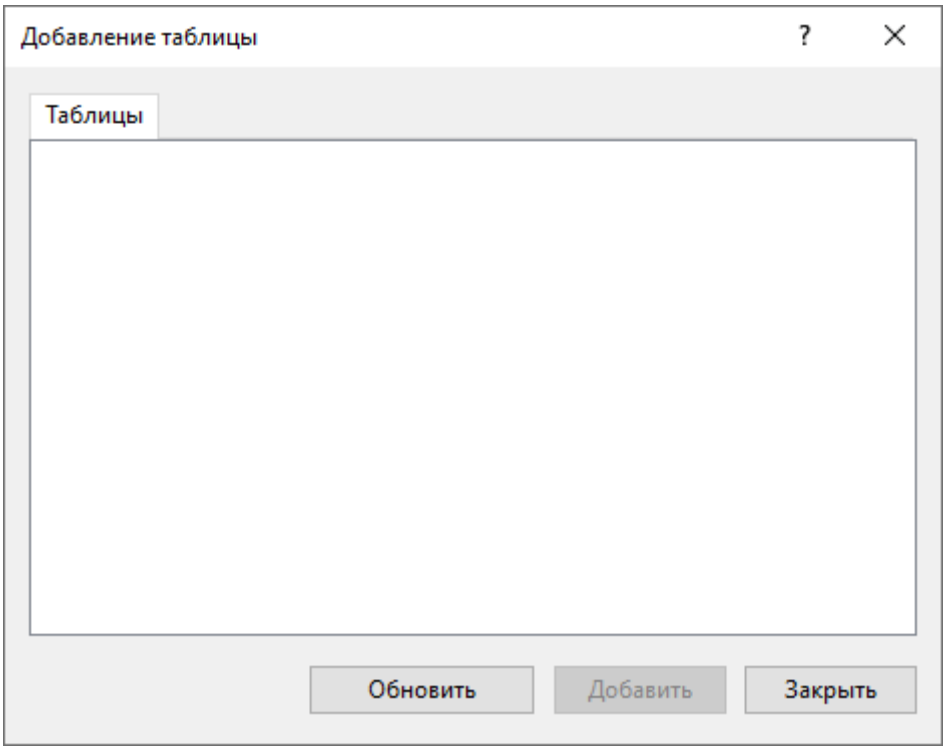
Для того чтобы создать диаграмму базы данных необходимо правой кнопкой по папке Диаграммы баз данных выбрать пункт Создать диаграмму баз данных



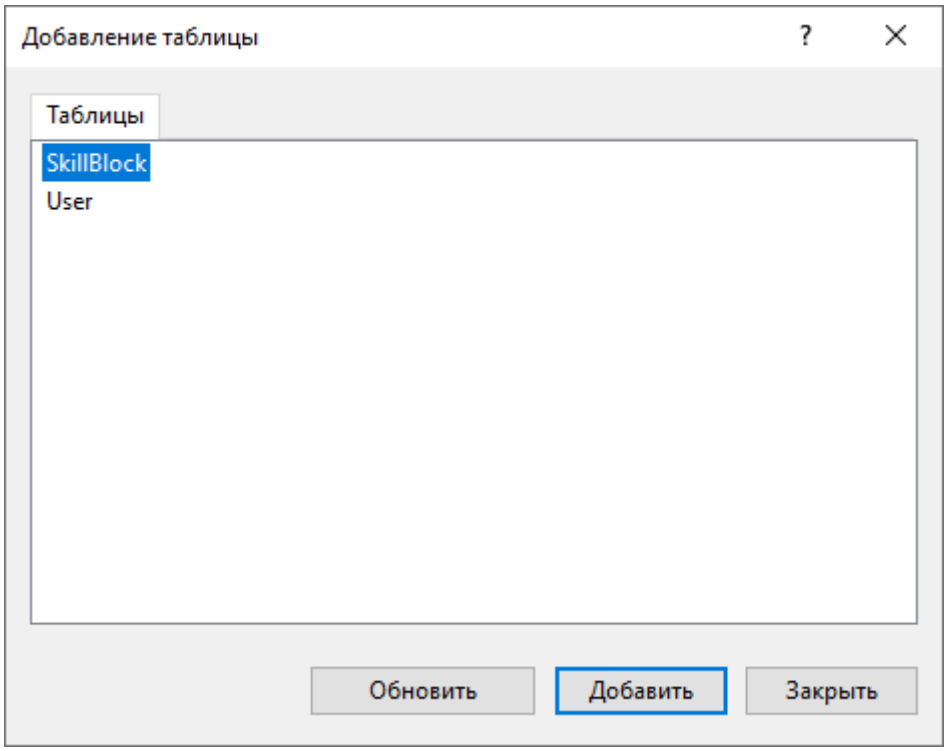
При создании первой диаграммы может выйти сообщение, его надо закрыть, нажав на кнопку Да.



После создания диаграммы открывается редактор, в котором откроется окно для добавления существующих таблиц, если таблиц в БД нет, то это окно будет пустым и его можно закрыть.



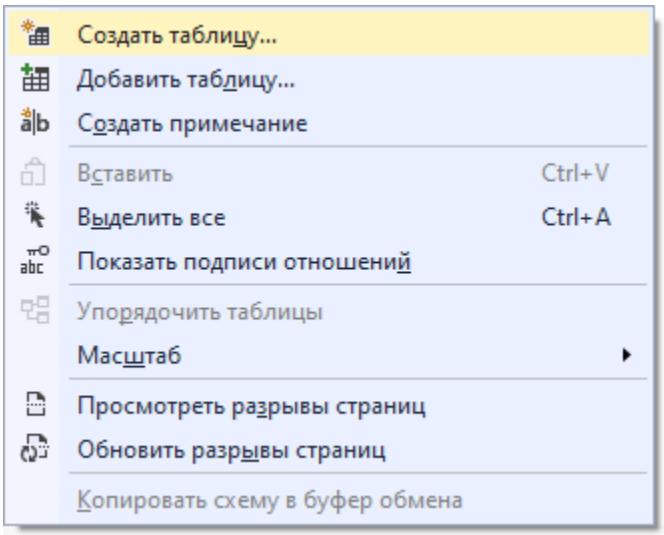
В случае если уже существуют таблицы в БД их необходимо выделить и нажать на кнопку Добавить, после этого они отобразятся на диаграмме.



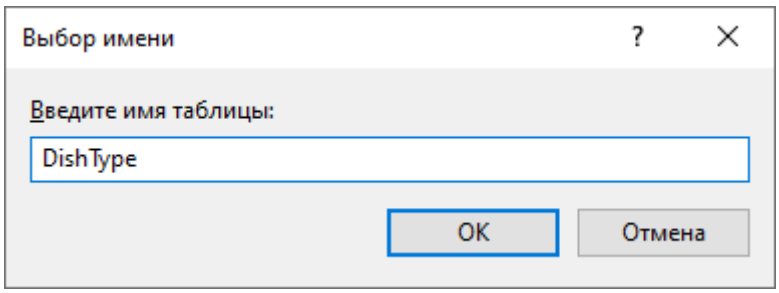
Добавление таблицы на диаграмму

Чтобы создать таблицу в диаграмме нужно:

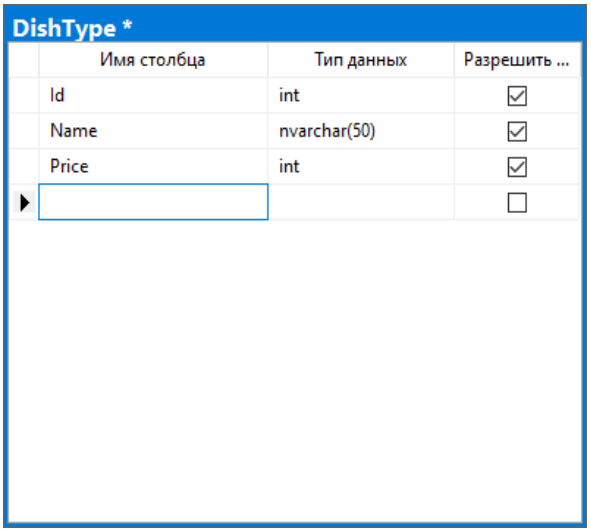
1. По свободному месту на диаграмме нажать правой кнопкой и выбрать пункт «Создать таблицу»



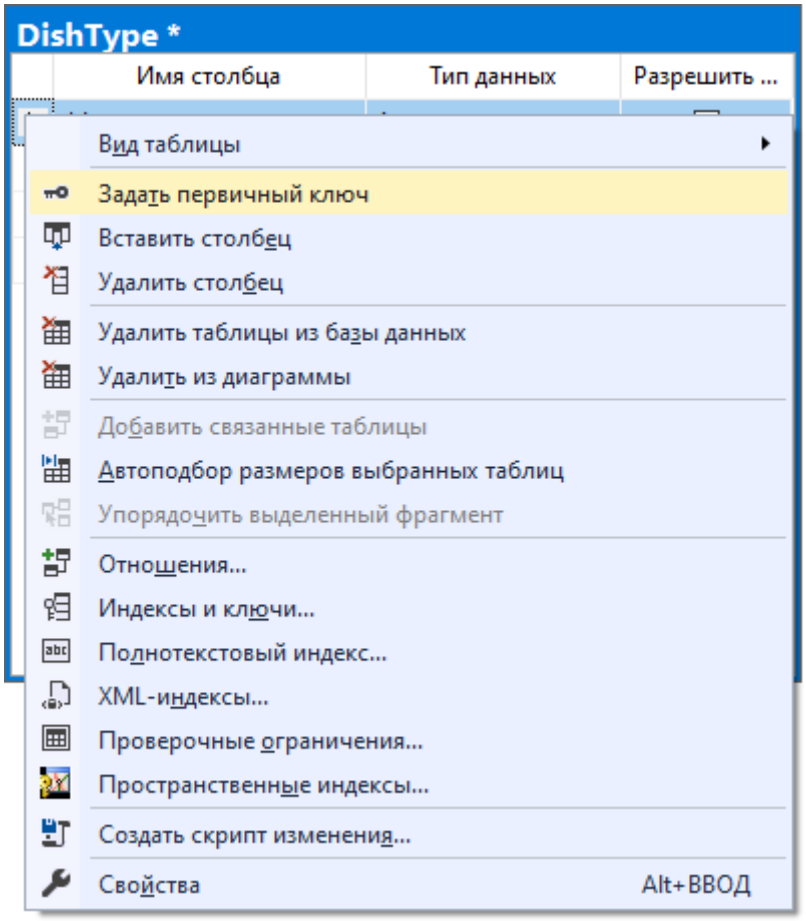
2. В открывшемся окне ввести название таблицы (На английском, без пробелов, в единственном числе)



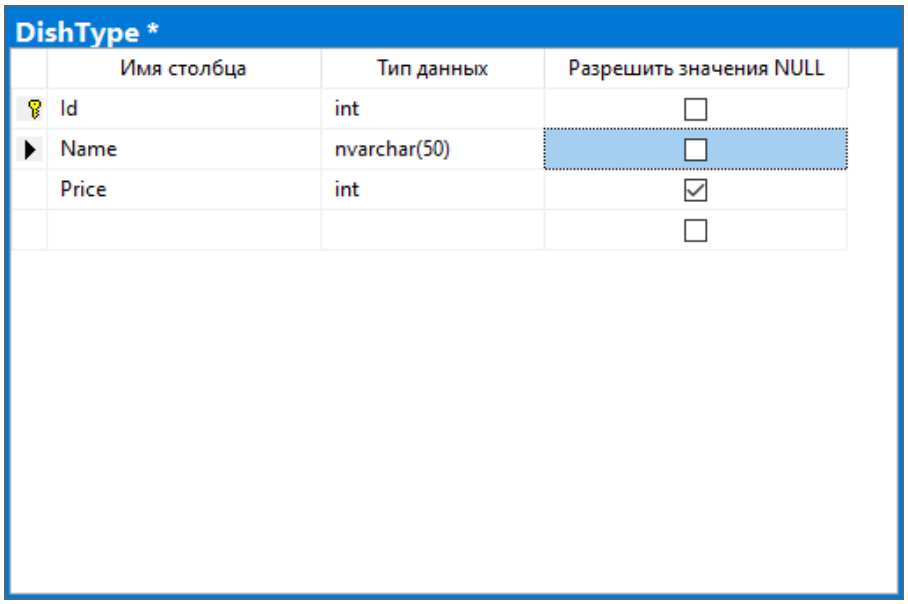
3. После создания таблицы заполнить атрибуты, типы данных и установить первичный ключ



Первичным ключом как правило выступает атрибут Id с типом данных int. Чтобы установить его как первичный нужной нажать по нему правой кнопкой и выбрать «Задать первичный ключ».



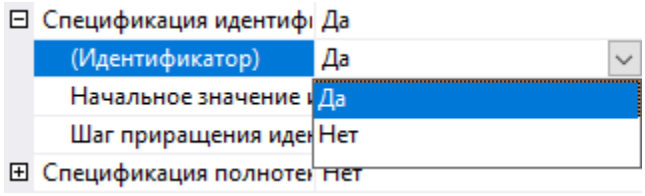
Также необходимо не забывать правильно указывать «Необязательность полей», в столбце «Разрешить значения NULL» (например, атрибут Name не должен разрешать значение NULL, поэтому галочку нужно убрать).



Обязательно также указать **идентификатор** для первичного ключа для этого

1. Правой кнопкой по первичному ключу открыть свойства

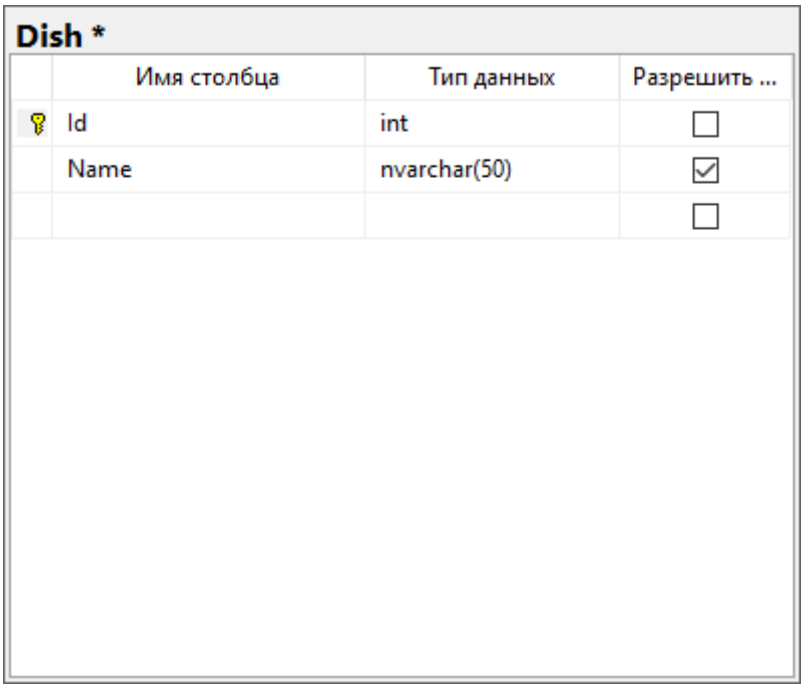
2. Справа в списке свойств открыть пункт Спецификация идентификатора и установить значение (Идентификатор) с “нет” на “да”



Создание связей между таблицами

Чтобы реализовать связь между двумя таблицами, одна из них должна иметь соответствующий внешний ключ. Внешний ключ — это значение первичного ключа другой таблицы. Поэтому атрибут первичного ключа должен иметь такой же тип данных, как и первичный ключ таблицы, с которой необходимо связаться.

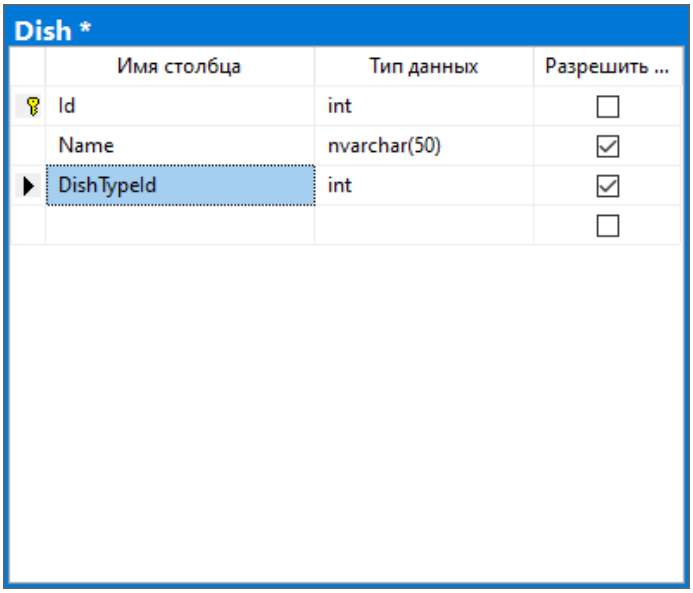
Создадим таблицу Dish (блюдо) и свяжем ее с таблицей DishType (тип блюда).



Для этого в таблицу Dish надо добавить внешний ключ чтобы связать ее с таблицей DishType. Так как типов блюд ограниченное количество, а блюд может быть гораздо больше, и они могут дополняться

внешний ключ добавляется именно в таблицу Dish. Такая связь называется “Один ко Многим” (Один тип блюда может быть во многих блюдах (проговаривайте подобное выражения, когда реализуете связи)).

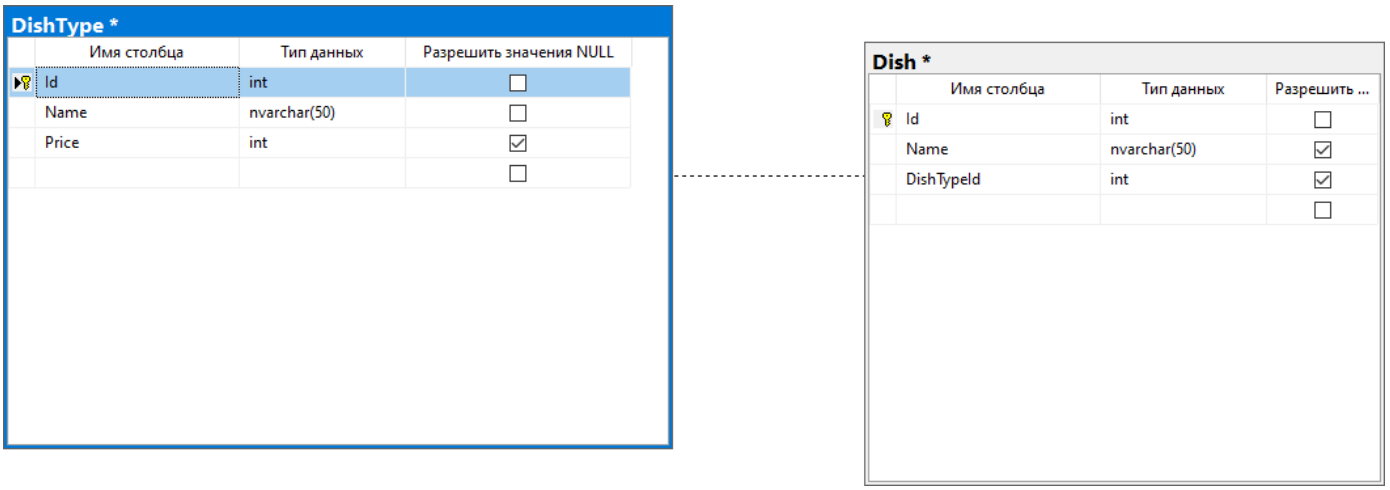
Как правило название атрибута внешнего ключа называется названием связываемой таблицы + Id



После того как внешний ключ был создан необходимо протянуть связь

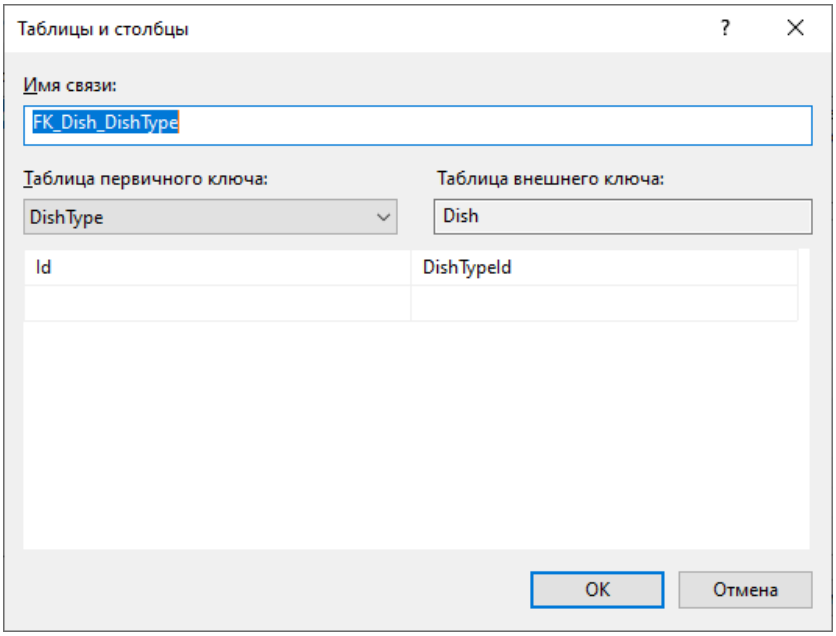
1. Зажать левой кнопкой первичный ключ таблицы

2. Протянуть связь к внешнему ключу другой таблицы (указатель должен быть строго на внешнем ключе)

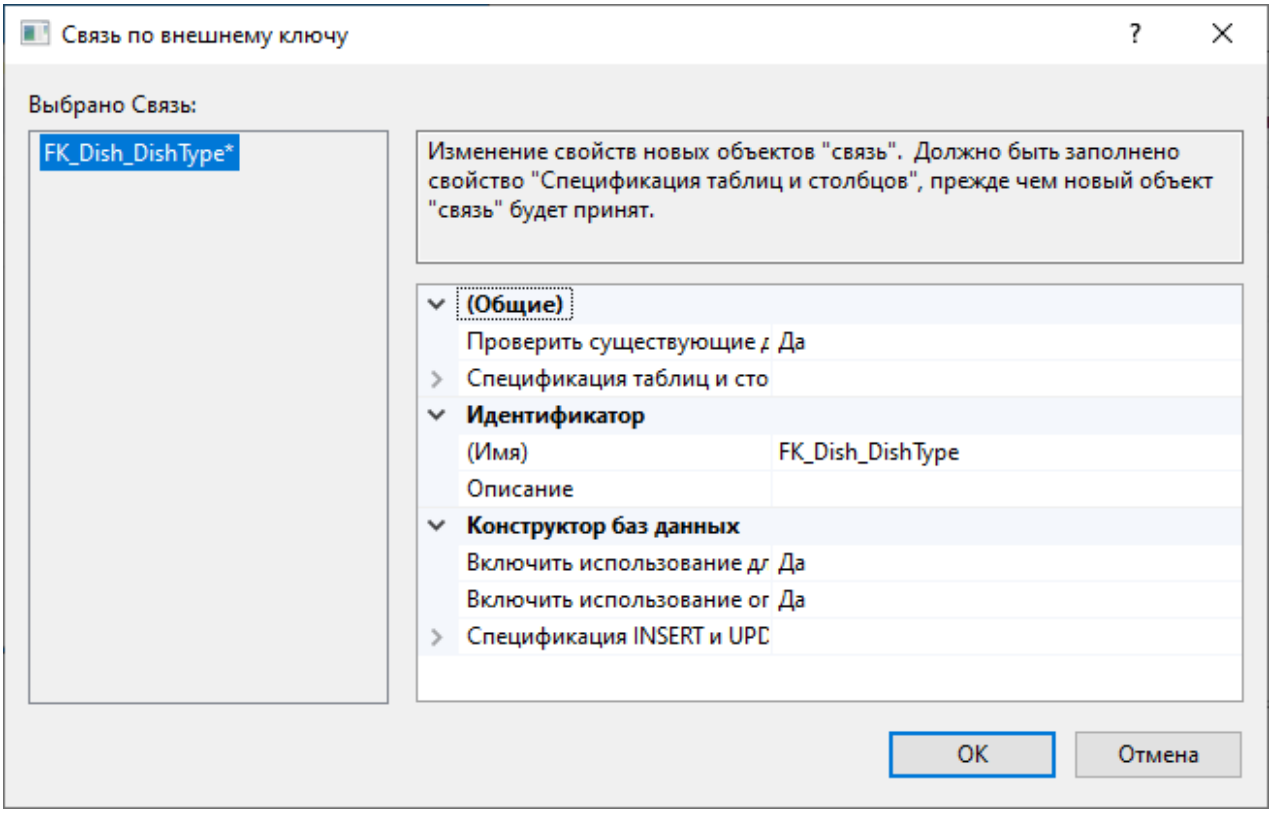


3. Отпустить левую кнопку

4. В появившемся окне проверить правильно ли указана (Слева должен быть первичный ключ, а справа внешний) связь и нажать Ок



5. Нажать Ок

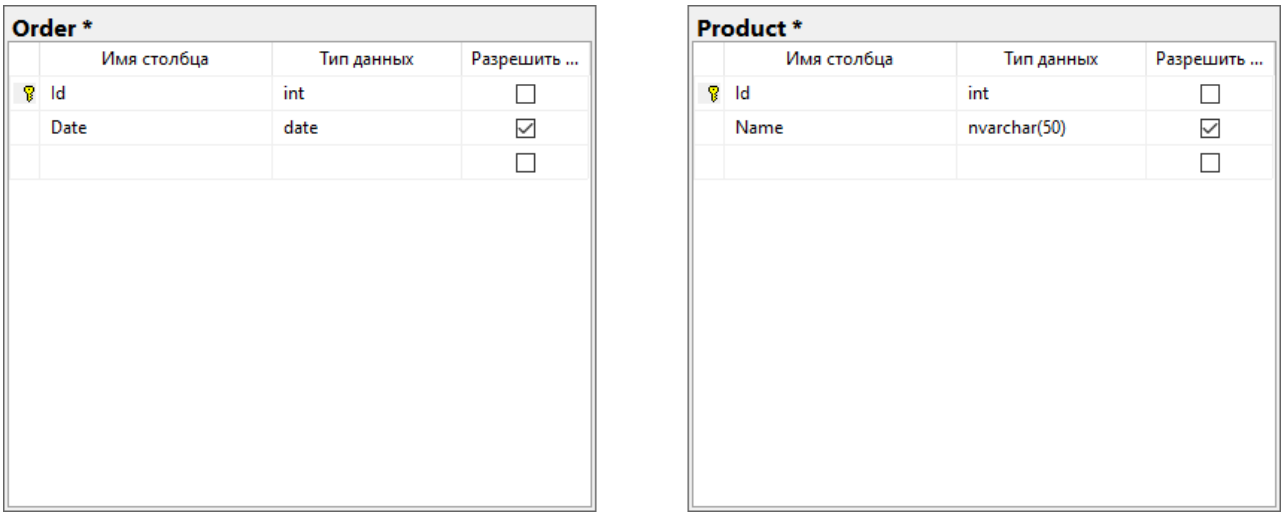


6. Связь создана. Ее можно двигать как вам угодно для удобного чтения



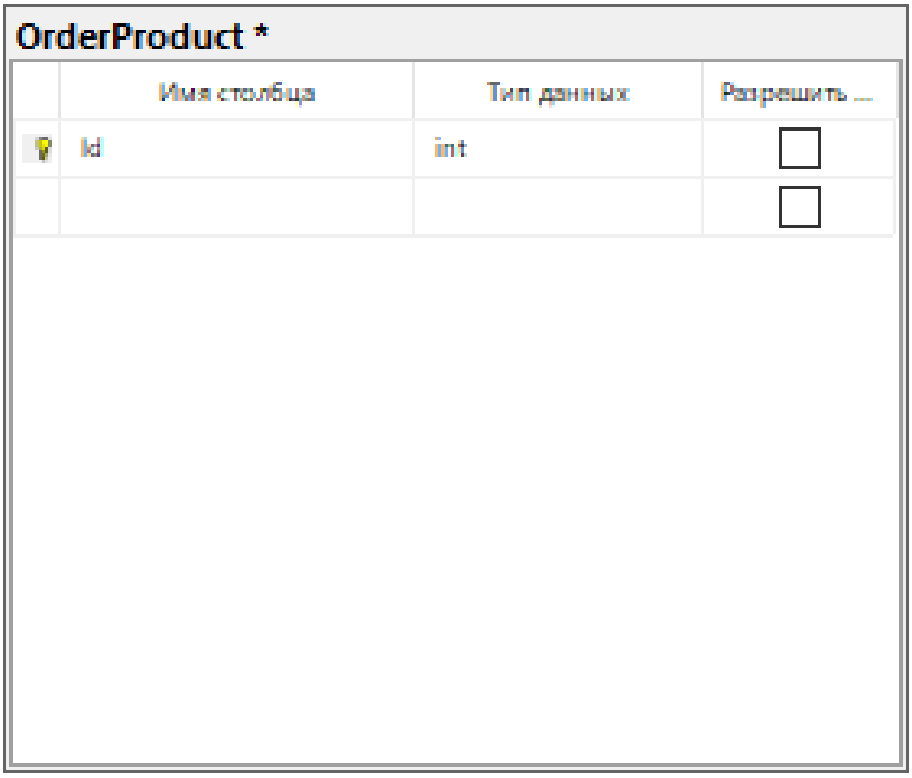
Создание связи “Много ко Многим”

Связь “Много ко Многим” возникает тогда, когда записи из первой таблицы соответствует много записи из второй и наоборот. Например, связь между таблицами Заказ и Продукт. У одного заказа может быть много продуктов и продукты могут быть во многих заказах.

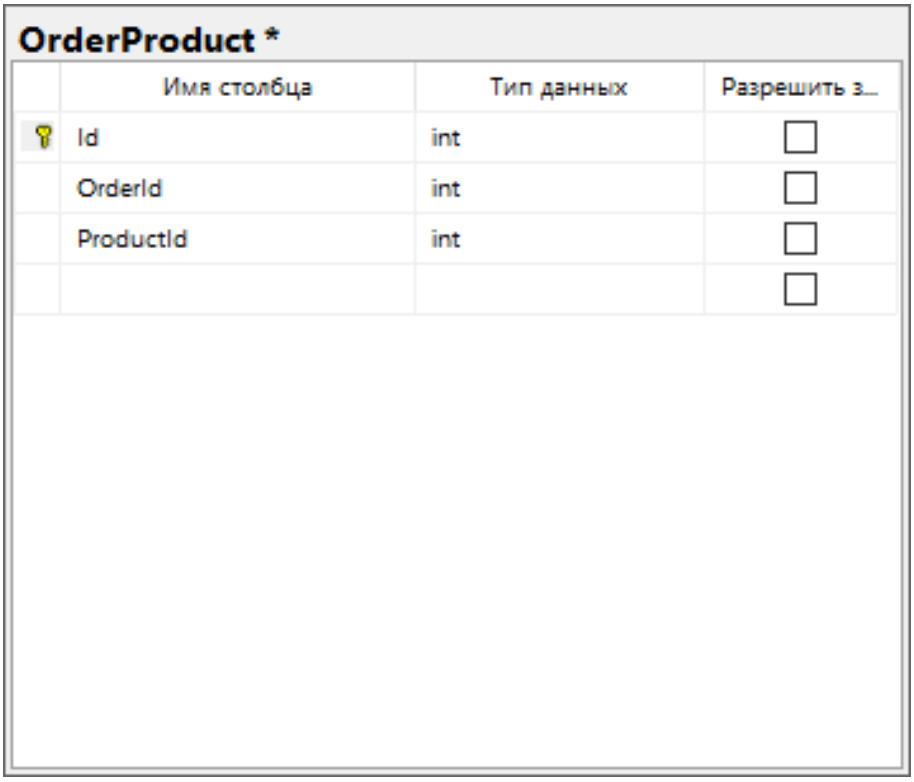


Для реализации связи “Много ко Многим” необходима дополнительная таблица, которая будет выступать в роли связи двух таблиц.

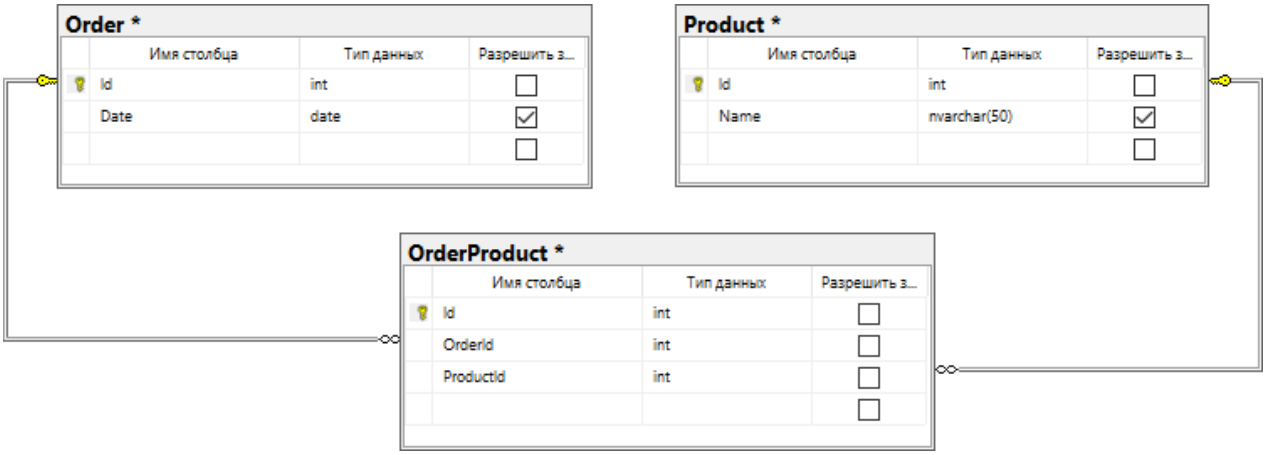
Как правило такую таблицу называют слитно двумя названиями таблиц, которые она связывает.



Главными атрибутами такой таблицы являются Первичный ключ и два внешних ключа связываемых таблиц.



И остается только провести связи от первичных ключей к внешним ключам как описано выше.



Теперь мы можем хранить продукты по каждому заказу.

Git

Git применяется для управления версиями в рамках колоссального количества проектов по разработке ПО, как коммерческих, так и с открытым исходным кодом.

Для работы с Git в этом пособии будет использоваться GitHub, следовательно, необходимо зарегистрировать учетную запись.

Установка Git

Для работы с Git необходимо его скачать. В процессе установки можно везде нажимать далее.

После успешной установки в контекстном меню появится два пункта - Git GUI Here и Git Bash Here.

Работа с Git Bash

Основные команды

- git init
- git add .
- git commit -m ”Сообщение”
- git push
- git clone ССылкаНаРепозиторий

Клонирование репозитория

Клонирование позволяет создать копию репозитория без его инициализации для дальнейшей работы с ним.

1. Перейти в директорию куда нужно клонировать репозиторий
2. Открыть консоль Git Bash
3. Скопировать URL нужного репозитория
4. В консоли прописать команду git clone и вставить скопированный URL
5. Выполнить команду
6. Убедиться, что репозиторий склонирован

Инициализация репозитория

Инициализация позволяет создать репозиторий в существующей папке (допустим был создан проект и его необходимо залить на Git).

1. Перейти в папку, где необходимо создать репозиторий
2. Открыть консоль Git Bash
3. Выполнить команду git init
4. Скопировать URL нужного репозитория
5. Выполнить команду git remote add origin и вставить скопированный URL

После этого в папке будет находится репозиторий с которым можно работать.

Отправка изменений

Какие-либо изменения содержимого репозитория не сохраняются на сервере автоматически. Поэтому следует выполнять фиксацию этих изменений и отправку их на сервер.

1. Открыть папку репозитория
2. Открыть консоль Git Bash
3. Выполнить команду git add .
4. Выполнить команду git commit -m “какое-то сообщение”
5. Выполнить команду git push

Личные рекомендации

Проще и быстрее всего клонировать существующий репозиторий на ПК и в дальнейшем с ним работать.

1. Создать репозиторий на GitHub
2. Склонировать на ПК
3. Создать в репозитории приложение/добавить файлы и т.д.
4. Фиксировать изменения

Работа с Git через Visual Studio

Для того чтобы выполнять отправку изменений на сервер через Visual Studio проект должен уже находится в репозитории.

1. Открыть проект в Visual Studio

2. Справа, где находится обозреватель решений есть вкладка Изменения Git. Если вкладка отсутствует, то ее необходимо добавить через Вид - Изменения Git.

3. Во вкладке Изменения Git если там присутствуют изменения нужно ввести сообщение (аналог атрибута -m “”)

4. Нажать на “Зафиксировать все” (аналог commit)

5. Нажать на “Отправить” (аналог push)

Измененные файлы в Visual Studio отображаются красной галочкой. Новые файлы в Visual Studio отображаются зеленым крестиком.

Зафиксированные файлы в Visual Studio отображаются синим замочком.

Работа с Git через GitHub Desktop (нескоро)

Часто встречаемые вопросы и проблемы

В этой главе рассматриваются проблемы, которые возникают при разработке проекта и имеют неявный характер ее решения.